COURS 3 — Les Structures Répétitives (boucles) en C

Par Robert DIASSÉ

1. Introduction aux boucles

Une boucle permet de **répéter** une ou plusieurs instructions **tant qu'une condition est vraie**, ou **pendant un nombre de fois déterminé**.

Les boucles évitent de répéter manuellement des instructions.

Exemples de situations adaptées aux boucles :

- Afficher les nombres de 1 à 10
- Lire plusieurs valeurs
- Parcourir un tableau
- Compter, calculer, répéter une action tant qu'une condition n'est pas atteinte

---

2. La boucle `for`

La boucle `for` permet de répéter une instruction **un nombre connu de fois**.

Format général :

```
for (initialisation ; condition ; incrémentation) {
    instructions;
}
```

Les trois parties jouent un rôle bien précis :

1. **Initialisation** : crée et initialise une variable (souvent un indice).
2. **Condition** : la boucle continue tant que cette condition est vraie.
3. **Incrémentation** : modification automatique de l'indice à chaque tour.

2.1 Exemple : afficher les nombres de 1 à 5

```
for (int i = 1; i <= 5; i++) {
    printf("%d ", i);
}
```

Explication :

- `int i = 1` → début à 1
- `i <= 5` → répéter tant que i ≤ 5
- `i++` → augmente i de 1 à chaque tour

Affichage :

```
1 2 3 4 5
```

2.2 Exemple : afficher les nombres pairs de 2 à 10

```
for (int i = 2; i <= 10; i += 2) {
    printf("%d ", i);
}
```

2.3 Exemple : descendre de 10 à 1

```
for (int i = 10; i >= 1; i--) {
    printf("%d ", i);
}
```

2.4 Importance de l'indice

L'indice est la **variable de contrôle** de la boucle :

- commence à une valeur
- évolue à chaque tour
- permet de compter, parcourir, répéter

Une boucle `for` **nécessite toujours un indice bien défini et mis à jour**.

Exercices – boucle `for`

1. Afficher les entiers de 1 à 20.
2. Afficher les multiples de 5 entre 5 et 50.

3. Lire un entier `n` et afficher les `n` premiers entiers.
4. Lire un entier `n` et afficher la table de multiplication de `n`.
5. Calculer la somme des entiers de 1 à `n`.

---

3. La boucle `while`

La boucle `while` répète une instruction **tant qu'une condition est vraie**.

Format général :

```
while (condition) {
    instructions;
}
```

L'exécution démarre **seulement si la condition est vraie dès le début**.

3.1 Exemple : afficher les nombres de 1 à 5

```
int i = 1;

while (i <= 5) {
    printf("%d ", i);
    i++;
}
```

Ici, il faut **manuellement** :

- déclarer l'indice (`int i = 1`)
- mettre à jour l'indice (`i++`)

3.2 Exemple : lecture répétée jusqu'à une valeur

```
int x;

printf("Entrez un nombre positif : ");
scanf("%d", &x);

while (x < 0) {
    printf("Erreur. Entrez un nombre positif : ");
    scanf("%d", &x);
}
```

La boucle continue tant que `x < 0` est vrai.

3.3 Exemple : afficher uniquement les pairs jusqu'à 20

```
int i = 2;

while (i <= 20) {
    printf("%d ", i);
    i += 2;
}
```

Exercices – boucle `while`

1. Lire un entier strictement positif, recommencer jusqu'à ce qu'il soit valide.
2. Afficher les nombres de 10 à 1 à l'aide d'un `while`.
3. Compter combien de fois l'utilisateur tape un nombre négatif avant de taper 0.
4. Lire des notes et arrêter dès qu'une note > 20 est saisie.
5. Afficher la somme des entiers de 1 à `n` avec un `while`.

---

4. La boucle `do … while`

Contrairement à `while`, une boucle `do … while` :

→ **exécute le bloc une première fois**,
→ vérifie la condition ensuite.

Format général :

```
do {
    instructions;
} while (condition);
```

Elle garantit **au moins une exécution**.

4.1 Exemple : demander un nombre positif

```
int x;

do {
    printf("Entrez un nombre positif : ");
    scanf("%d", &x);
} while (x < 0);
```

Ici, même si le nombre est déjà correct, la saisie s'exécute une fois.

4.2 Exemple : menu simple

```
int choix;

do {
    printf("1: Bonjour\n");
    printf("2: Au revoir\n");
    printf("3: Quitter\n");
    printf("Votre choix : ");
    scanf("%d", &choix);
} while (choix != 3);
```

4.3 Exemple : compter les chiffres d'un nombre

```
int n, cpt = 0;

printf("Entrez un entier : ");
scanf("%d", &n);

do {
    n /= 10;
    cpt++;
} while (n != 0);

printf("Nombre de chiffres : %d\n", cpt);
```

Exercices – boucle `do...while`

1. Proposer un menu et répéter tant que l'option choisie n'est pas 0.
2. Lire un entier et répéter la saisie tant qu'il est négatif.
3. Lire un caractère puis recommencer tant qu'il n'est pas une voyelle.
4. Lire 5 notes avec un `do...while` en utilisant un compteur.
5. Lire un nombre et compter ses chiffres.

---

5. Différences entre `for`, `while`, et `do...while`

Boucle	Commence quand ?	Condition vérifiée quand ?	Utilisation typique

Boucle	Commence quand ?	Condition vérifiée quand ?	Utilisation typique
`for`	avant la boucle	avant le bloc	nombre **défini** de répétitions
`while`	avant la boucle	avant le bloc	répéter **tant que** une condition reste vraie
`do...while`	toujours une fois	après le bloc	répéter jusqu'à respecter la condition

Résumé :

- `for` → nombre de répétitions connu
- `while` → tant que condition vraie
- `do...while` → au moins une exécution

---

6. Exercices finaux (synthèse des trois boucles)

Exercice A

Lire un entier `n` et afficher les entiers de 1 à `n` avec un `for`.

Exercice B

Lire un entier positif, recommencer jusqu'à ce qu'il soit valide (`while`).

Exercice C

Créer un menu :

```
1 - Afficher bonjour
2 - Afficher un nombre aléatoire
3 - Quitter
```

Utiliser `do...while`.

Exercice D

Calculer la somme des entiers de 1 à `n` en utilisant :

- une version `for`
- une version `while`

Exercice E

Afficher tous les multiples de 3 entre 0 et 50.